

//1. Create a base class with the following members:

//- Private integer privateInt

//- Protected integer protectedInt

//- Public integer publicInt

//Create getters and setters for each of these variables. Derive 3 classes from the base

//class with the three types of inheritance based on visibility(public, protected, private).

//You can name these classes as publicChild, privateChild and protectedChild. After

//doing this, try and figure out which member you can access publicly or through getters

//and setters. Then print out the way that you were able to access them. For example, if

//you did private inheritance, you could not be able to access the members in the child

//classes, and would need to use their getters or setters.

```
#include<iostream>
```

```
using namespace std;
```

```
class Base{
```

```
    private:
```

```
        int privateInt;
```

```
    protected:
```

```
        int protectedInt;
```

```
    public:
```

```
        int publicInt;
```

```
    //making setters
```

```
    void setprivateInt(int privateInt)
```

```
    {
```

```
        this->privateInt=privateInt;
```

```
    }
```

```
    void setprotectedInt(int protectedInt)
```

```
    {
```

```
        this->protectedInt=protectedInt;
```

```
    }
```

```
    void setpublicInt(int publicInt)
```

```
    {
```

```
        this->publicInt=publicInt;
```

```
    }
```

```
    //making getters
```

```
    int getprivateInt()
```

```
    {
```

```
        return privateInt;
```

```
    }
```

```
    int getprotectedInt()
```

```
    {
```

```
        return protectedInt;
```

```
    }
```

```
    int getpublicInt()
```

```
    {
```

```
        return publicInt;
```

```

    }
};
class publicChild: public Base{
    public:
        void showaccess()
        {
            setprivateInt(1);
            protectedInt=2;
            publicInt=3;
            cout<<"Printing!"<<endl;
            cout<<"Private int: "<<getprivateInt()<<"\nAccessed through getter and
setter!"<<endl;
            cout<<"Protected int: "<<protectedInt<<"\nDirectly accessed!"<<endl;
            cout<<"Public int: "<<publicInt<<"Directly accessed!"<<endl;
        }
};
class protectedChild: protected Base{
    public:
        void showaccess()
        {
            setprivateInt(4);
            protectedInt=5;
            publicInt=6;
            cout<<"Printing!"<<endl;
            cout<<"Private int: "<<getprivateInt()<<"\nAccessed through getter and setter
while setting and printing!"<<endl;
            cout<<"Protected int: "<<protectedInt<<"\nDirectly accessed while setting and
printing!"<<endl;
            cout<<"Public int: "<<publicInt<<"\nDirectly accessed while setting and
printing!"<<endl;
        }
};
class privateChild: private Base{
    public:
        void showaccess()
        {
            setprivateInt(7);
            protectedInt=8;
            publicInt=9;
            cout<<"Printing!"<<endl;
            cout<<"Private int: "<<getprivateInt()<<"\nAccessed through getter and setter
while setting and printing!"<<endl;
            cout<<"Protected int: "<<protectedInt<<"\nDirectly accessed while setting and
printing!"<<endl;
            cout<<"Public int: "<<publicInt<<"\nDirectly accessed while setting and
printing!"<<endl;
        }
};

```

```

int main()
{
    publicChild obj1;
    obj1.publicInt=100;
    cout<<"\nFor the object of publicChild we can directly access publicInt but for
protectedInt and privateInt , foe them we would have to use setters and getters!";
    cout<<"\nPublic inheritance!"<<endl;
    obj1.showaccess();

    protectedChild obj2;
    cout<<"\nFor the object of protectedChild we can directly access none and  for
protectedInt and privateInt and publicInt, foe them we would have to use setters and
getters!";
    cout<<"\nProtected inheritance!"<<endl;
    obj2.showaccess();

    privateChild obj3;
    cout<<"\nFor the object of protectedChild we can directly access none and  for
protectedInt and privateInt and publicInt, foe them we would have to use setters and
getters!";
    cout<<"\nPrivate inheritance!"<<endl;
    obj3.showaccess();
}

```

OUTPUT:

```
Accessed through getter and setter!
Protected int: 2
Directly accessed!
Public int: 3Directly accessed!

For the object of protectedChild we can directly access none and for protectedInt and privateInt and publicInt, for the
m we would have to use setters and getters!
Protected inheritance!
Printing!
Private int: 4
Accessed through getter and setter while setting and printing!
Protected int: 5
Directly accessed while setting and printing!
Public int: 6
Directly accessed while setting and printing!

For the object of protectedChild we can directly access none and for protectedInt and privateInt and publicInt, for the
m we would have to use setters and getters!
Private inheritance!
Printing!
Private int: 7
Accessed through getter and setter while setting and printing!
Protected int: 8
Directly accessed while setting and printing!
Public int: 9
Directly accessed while setting and printing!

-----
Process exited after 3.044 seconds with return value 0
Press any key to continue . . . |
```

//2.Create a class Teacher with the following private attributes: Name,
//Age, and Institute.
//Derive three classes from it that has the following names: HumanitiesTeacher,
// ScienceTeacher,
//MathsTeacher. These classes should have the following members:
//- Department (this should have the value “science”, “maths” or “humanities”)
//- Course Name
//- Designation (for example, lecturer, professor, etc)
//Create proper accessors and mutators for the attributes. Create objects for each of the
//classes and display the values. You can ask the user to input the values.

```
#include<iostream>
using namespace std;
class Teacher{
    string name;
    int age;
    string Institute;
public:
    //getters
    string getname()
    {
        return name;
    }
    int getage()
    {
        return age;
    }
    string getInstitute()
    {
        return Institute;
    }

    //setters
    void setname(string name)
    {
        this->name=name;
    }
    void setage(int age)
    {
        this->age=age;
    }
    void setInstitute(string Institute)
    {
        this->Institute=Institute;
    }
};
class HumanitiesTeacher: public Teacher{
    const string Department;
    string Course_Name;
```

```

string Designation;

public:
HumanitiesTeacher(): Department("Humanities")
{
}
//making setters

void setCourseName(string cn)
{
Course_Name=cn;
}
void desig(string desig)
{
    Designation=desig;
}

//making getters
string getDept()
{
    return Department;
}

string getcoursename()
{
    return Course_Name;
}

string getdesignation()
{
    return Designation;
}

void show()
{
    cout<<"\nDepartemnt: "<<Department<<endl;
    cout<<"\nCourse Name: "<<Course_Name<<endl;
    cout<<"\nDesignation: "<<Designation<<endl;
    cout<<"\nName: "<<getname()<<endl;
    cout<<"\nAge: "<<getage()<<endl;
    cout<<"\nInstitute: "<<getInstitute();

}

void input()
{
    cout<<"\nEnter the Course Name: "<<endl;
    cin>>Course_Name;
}

```

```

        cout<<"\nEnter the designation: "<<endl;
        cin>>Designation;
        cout<<"\nEnter the name: "<<endl;
        string nm;
        cin>>nm;
        setname(nm);
        cout<<"\nEnter the age: "<<endl;
        int ag;
        cin>>ag;
        setage(ag);

        cout<<"\nEnter the institute: "<<endl;
        string ln;
        cin>>ln;
        setInstitute(ln);
    }
};

class ScienceTeacher: public Teacher{
    const string Department;
    string Course_Name;
    string Designation;

public:
    ScienceTeacher(): Department("Science")
    {
    }
    //making setters

    void setCourseName(string cn)
    {
        Course_Name=cn;
    }
    void desig(string desig)
    {
        Designation=desig;
    }

    //making getters
    string getDept()
    {
        return Department;
    }

    string getcoursename()
    {
        return Course_Name;
    }
}

```

```

string getdesignation()
{
    return Designation;
}

void show()
{
    cout<<"\nDepartemnt: "<<Department<<endl;
    cout<<"\nCourse Name: "<<Course_Name<<endl;
    cout<<"\nDesignation: "<<Designation<<endl;
    cout<<"\nName: "<<getname()<<endl;
    cout<<"\nAge: "<<getage()<<endl;
    cout<<"\nInstitute: "<<getInstitute();
}

void input()
{
    cout<<"\nEnter the Course Name: "<<endl;
    cin>>Course_Name;
    cout<<"\nEnter the designation: "<<endl;
    cin>>Designation;
    cout<<"\nEnter the name: "<<endl;
    string nm;
    cin>>nm;
    setname(nm);
    cout<<"\nEnter the age: "<<endl;
    int ag;
    cin>>ag;
    setage(ag);

    cout<<"\nEnter the institute: "<<endl;
    string In;
    cin>>In;
    setInstitute(In);
}
};

class MathsTeacher: public Teacher{
    const string Department;
    string Course_Name;
    string Designation;

public:
    MathsTeacher(): Department("Maths")
    {
    }

    //making setters

```



```

void setCourseName(string cn)
{
    Course_Name=cn;
}
void desig(string desig)
{
    Designation=desig;
}

//making getters
string getDept()
{
    return Department;
}

string getcoursename()
{
    return Course_Name;
}

string getdesignation()
{
    return Designation;
}

void show()
{
    cout<<"\nDepartemnt: "<<Department<<endl;
    cout<<"\nCourse Name: "<<Course_Name<<endl;
    cout<<"\nDesignation: "<<Designation<<endl;
    cout<<"\nName: "<<getname()<<endl;
    cout<<"\nAge: "<<getage()<<endl;
    cout<<"\nInstitute: "<<getInstitute();

}

void input()
{
    cout<<"\nEnter the Course Name: "<<endl;
    cin>>Course_Name;
    cout<<"\nEnter the designation: "<<endl;
    cin>>Designation;
    cout<<"\nEnter the name: "<<endl;
    string nm;
    cin>>nm;
    setname(nm);
    cout<<"\nEnter the age: "<<endl;

```

```

        int ag;
        cin>>ag;
        setage(ag);

        cout<<"\nEnter the institute: "<<endl;
        string ln;
        cin>>ln;
        setInstitute(ln);
    }
};

int main()
{
    HumanitiesTeacher t1;
    cout<<"\nFor humanities group!: "<<endl;
    t1.input();
    t1.show();

    ScienceTeacher t2;
    cout<<"\nFor Science group!: "<<endl;
    t2.input();
    t2.show();

    MathsTeacher t3;
    cout<<"\nFor Maths group!: "<<endl;
    t3.input();
    t3.show();
}

```

OUTPUT:

```
For humanities group!:

Enter the Course Name:
honesty

Enter the designation:
lecturer

Enter the name:
khadija

Enter the age:
19

Enter the institute:
PECHS

Departemnt: Humanities

Course Name: honesty

Designation: lecturer

Name: khadija

Age: 19

Institute: PECHS
For Science group!:

Enter the Course Name:
CS

Enter the designation:
head

Enter the name:
Ali

Enter the age:
22

Enter the institute:
NED

Departemnt: Science

Course Name: CS

Designation: head

Name: Ali

Age: 22

Institute: NED
For Maths group!:

Enter the Course Name:
calculus

Enter the designation:
researcher

Enter the name:
bilal

Enter the age:
34

Enter the institute:
Metroplotan

Departemnt: Maths

Course Name: calculus

Designation: researcher

Name: bilal

Age: 34

Institute: Metroplotan
=====
Process exited after 244.6 seconds with return value 0
Press any key to continue . . . |
```

//3.A defense organization is making a hierarchy of different types of weapons. They have
//classified the nuclear bomb as follows: Weapons ? Hot Weapons ? Bombs ?
//Nuclear Bombs. Create classes and apply inheritance as necessary for the above
//hierarchy. Each class should have a method called: "xxxxxDescription", where xxxx
//would be class name. The method should print out what that weapon does. Eg. Hot
//weapons uses gunpowder, or explode. Bombs blow up. Nuclear bombs blow up, and
//use nuclear fission and fusion.

```
#include<iostream>
```

```
using namespace std;
```

```
class Weapons{
```

```
    public:
```

```
        void WeaponsDescription()
```

```
        {
```

```
            cout<<"\nWeapons are a cause of destruction!"<<endl;
```

```
        }
```

```
};
```

```
class HotWeapons: public Weapons{
```

```
    public:
```

```
        void HotWeaponsDescription()
```

```
        {
```

```
            cout<<"\n Hot weapons used gunpowder, or explode"<<endl;
```

```
        }
```

```
};
```

```
class Bombs: public HotWeapons{
```

```
    public:
```

```
        void BombsDescription()
```

```
        {
```

```
            cout<<"\nBombs blow up"<<endl;
```

```
        }
```

```
};
```

```
class Nuclear_Bombs: public Bombs{
```

```
    public:
```

```
        void Nuclear_BombsDescription()
```

```
        {
```

```
            cout<<"\nNuclear bombs use nuclear fusion and fission"<<endl;
```

```
        }
```

```
};
```

```
int main()
```

```
{
```

```
    Nuclear_Bombs NB;
```

```
    cout<<"\nThe complete discription of a nuclear bomb:"<<endl;
```

```
    NB.WeaponsDescription();
```

```
    NB.HotWeaponsDescription();
```

```
    NB.BombsDescription();
```

```
    NB.Nuclear_BombsDescription();
```

```
    Bombs B;
```

```
    cout<<"\nThe complete discription of a bomb:"<<endl;
```

```
B.WeaponsDescription();  
B.HotWeaponsDescription();  
B.BombsDescription();
```

```
cout<<"\nThe complete discription of a hot weapon:"<<endl;  
HotWeapons HW;  
HW.WeaponsDescription();  
HW.HotWeaponsDescription();
```

```
cout<<"\nThe complete discription a of weapon:"<<endl;  
Weapons W;  
W.WeaponsDescription();
```

```
}
```

OUTPUT:

```
The complete discription of a nuclear bomb:
Weapons are a cause of destruction!
    Hot weapons used gunpowder, or explode
Bombs blow up
Nuclear bombs use nuclear fusion and fission
The complete discription of a  bomb:
Weapons are a cause of destruction!
    Hot weapons used gunpowder, or explode
Bombs blow up
The complete discription of a hot weapon:
Weapons are a cause of destruction!
    Hot weapons used gunpowder, or explode
The complete discription a of weapon:
Weapons are a cause of destruction!
-----
Process exited after 2.905 seconds with return value 0
Press any key to continue . . . |
```

//4. A bakery is making a program to calculate the bills for each of their customers. The items in the bakery are categorized as follows:

//Item:

//- Name

//- Quantity

//Baked Goods: (derived from item)

//- Discount = 10%

//Cakes: (derived from baked goods)

//- Price = 600

//Bread: (derived from baked goods)

//- Price = 200

//Drinks: (derived from item)

//- Discount = 5%

//- Price = 100

//In your main function, as a bakery personnel, input the items and their quantities and

//calculate the bill accordingly.

```
#include<iostream>
```

```
using namespace std;
```

```
class Item{
```

```
    public:
```

```
        string name;
```

```
        int quantity;
```

```
};
```

```
class BakedGoods: public Item{
```

```
    public:
```

```
        const float discount;
```

```
        BakedGoods(): discount(0.1)
```

```
        {
```

```
        }
```

```
};
```

```
class Cakes: public BakedGoods{
```

```
    public:
```

```
        const int price;
```

```
        Cakes(): price(600)
```

```
        {
```

```
        }
```

```
};
```

```
class Bread: public BakedGoods{
```

```
    public:
```

```
        const int price;
```

```
        Bread():price(200)
```

```
        {
```

```
        }
```

```
};
```

```
class Drinks: public Item{
```

```
    public:
```

```
        const float discount;
```

```
        const int price;
```

```

        Drinks(): discount(0.05),price(100)
        {
        }
};

int main()
{
    int a;
    float totalbill=0;
    Cakes c1;
    Bread b1;
    Drinks d1;

    while(a!=4)
    {
        cout<<"\nEnter 1 for Cakes:"<<endl;
        cout<<"\nEnter 2 for Breads:"<<endl;
        cout<<"\nEnter 3 for Drinks:"<<endl;
        cout<<"\nEnter 4 to exit:"<<endl;

        cin>>a;
        cout<<"\nYour choice: "<<a<<endl;

        switch(a)
        {
            case 1:
                cout<<"\nSo,for cake:"<<endl;
                cout<<"\nBy the way the price of cakes is 600 and there is a discount
of 10% on it!!"<<endl;
                cout<<"\nEnter the name: "<<endl;
                cin>>c1.name;
                cout<<"\nEnter the quantity: "<<endl;
                cin>>c1.quantity;

                totalbill+=((c1.quantity)*(c1.price))-((c1.quantity)*(c1.price)*(c1.discount));
                cout<<"\ntotalbill: "<<totalbill<<endl;
                break;

            case 2:
                cout<<"\nSo,for bread:"<<endl;
                cout<<"\nBy the way the price of breads is 200 and there is a discount
of 10% on it!!"<<endl;
                cout<<"\nEnter the name: "<<endl;
                cin>>b1.name;
                cout<<"\nEnter the quantity: "<<endl;
                cin>>b1.quantity;

```



```

totalbill+=((b1.quantity)*(b1.price))-((b1.quantity)*(b1.price)*(b1.discount));
        cout<<"\ntotalbill: "<<totalbill<<endl;
        break;

        case 3:
            cout<<"\nSo,for drinks:"<<endl;
            cout<<"\nBy the way the price of drinks is 100 and there is a discount
of 5% on it!!"<<endl;
            cout<<"\nEnter the name: "<<endl;
            cin>>d1.name;
            cout<<"\nEnter the quantity: "<<endl;
            cin>>d1.quantity;

totalbill+=((d1.quantity)*(d1.price))-((d1.quantity)*(d1.price)*(d1.discount));
        cout<<"\ntotalbill: "<<totalbill<<endl;
        break;

        default:
            cout<<"\nInvalid option :( "<<endl;
    }
}
cout<<"\nThe total bill is:"<<totalbill;
return 0;
}

```

OUTPUT:

```
Enter 1 for Cakes:
Enter 2 for Breads:
Enter 3 for Drinks:
Enter 4 to exit:
1
Your choice: 1
So,for cake:
By the way the price of cakes is 600 and there is a discount of 10% on it!!
Enter the name:
chocolate
Enter the quantity:
2
totalbill: 1080
Enter 1 for Cakes:
Enter 2 for Breads:
Enter 3 for Drinks:
Enter 4 to exit:
2
Your choice: 2
So,for bread:
By the way the price of breads is 200 and there is a discount of 10% on it!!
Enter the name:
brown_bread
Enter the quantity:
2
totalbill: 1440
Enter 1 for Cakes:
Enter 2 for Breads:
Enter 3 for Drinks:
Enter 4 to exit:
3
Your choice: 3
So,for drinks:
By the way the price of drinks is 100 and there is a discount of 5% on it!!
Enter the name:
pepsi
Enter the quantity:
2
totalbill: 1630
Enter 1 for Cakes:
Enter 2 for Breads:
Enter 3 for Drinks:
Enter 4 to exit:
4
Your choice: 4
Invalid option :(
The total bill is:1630
-----
Process exited after 78.27 seconds with return value 0
Press any key to continue . . . |
```